```
>>> A = torch.nn.functional.softmax(A, dim=3)
```

As you can see, we pass *Q* through a linear layer and *K* through its own linear layer, add them together, and then apply a nonlinear activation function. Then we pass this result through another linear layer and lastly apply the softmax across the node rows, which finally yields the attention weight tensor.

Now we do the same as before and contract the attention tensor with the *V* tensor along the last *n* dimension to get a tensor with dimensions $b \times h \times n \times d$, which is a multi-headed node matrix.

```
>>> E = torch.einsum('bhfc,bhcd->bhfd',A,V)
```

What we want at the end of the self-attention module is an updated node matrix with dimensions $b \times n \times d$, so we will concatenate or collapse the head dimension and *d* dimension, and then pass this through a linear layer to reduce the dimensionality back down to size *d*.

```
>>> E = rearrange(E, 'b head n d -> b n (head d)')
>>> E = self.linear1(E)
>>> E = torch.relu(E)
>>> E = self.norm1(E)
```

The final shape of this is now $b \times n \times d$, which is exactly what we want. Since we're only going to use a single self-attention module, we want to reduce this 3-tensor into a 2-tensor of just a batch of vectors, so we will maxpool over the *n* dimension, and then pass the result through a final linear layer, which represents the Q values.

```
>>> E = E.max(dim=1)[0]
>>> y = self.linear2(E)
>>> y = torch.nn.functional.elu(y)
```

That's it. We just went through all the core lines of code, but let's see it all together and test it out.

Listing 10.7. Multi-head relational module

```
class MultiHeadRelationalModule(torch.nn.Module):
    def __init__(self):
```